Operating Systems - Orange Problem

As you all likely know, (and if you're a non-player like me, don't worry we have you covered), *Clash Royale* is a real-time multiplayer game where players use a deck of cards to deploy various characters (**Troops**) onto a battlefield (**The Arena**). The goal is to destroy the opponent's main base (**The King Tower**).

To play a card and spawn a troop, a player must spend a magical, regenerating energy called **Elixir**.

For today, you are a backend systems engineer at Supercell (the game's developer). You are tasked with writing the core OS-level synchronization engine for a custom, automated multiplayer server.

Instead of human players, this custom server uses automated **Card Spawners**. A Spawner is simply an independent routine that repeatedly picks a card and tries to deploy it.

In your C program, these Card Spawners are your OS Threads. You will have multiple Spawner Threads running concurrently, constantly trying to spend Elixir to deploy different troops into the Arena.

However, a game server is a delicate environment. You must strictly manage the following OS concepts to keep the server from crashing:

1. Elixir Deadlocks (Resource Allocation & Deadlock Prevention):

The Game Rule: Cards cost different amounts of Elixir (e.g., a Goblin costs 2, a Knight costs 3). Elixir is a globally shared resource pool.

- **The OS Mapping:** What happens if a 4-Elixir Spawner Thread reserves 2 Elixir and waits for more, while a 3-Elixir Spawner Thread reserves the remaining 1 Elixir and waits? The server will **deadlock** (freeze forever). To prevent this, you must implement **Hold-and-Wait Prevention**. A Spawner Thread must secure its *full* Elixir cost simultaneously in one atomic action. If the server doesn't have enough Elixir right now, the thread must back off and wait.

2. The Arena Buffer (Synchronization & Bounded Buffers):

- **The Game Rule:** To prevent client-side lag and server crashes, the physical Arena can only hold a strict maximum number of active troops at any given time.

- **The OS Mapping:** The Arena is a **Bounded Buffer** (a shared array). Your Spawner Threads act as *Producers*. If the Arena is full, Spawner Threads must be put to sleep (wait) until a slot opens up (simulating a troop dying). You must use Semaphores and Mutexes to ensure two Spawners don't accidentally deploy troops into the exact same memory index at the exact same time (a Race Condition).

3. The Battle Log (Inter-Process Communication / IPC):

- **The Game Rule:** Players need a live text feed of what is happening in the match.
- **The OS Mapping:** You must create a separate Logger Thread. Whenever a Spawner successfully deploys a troop, it must send a message through a **Unidirectional Pipe** to the Logger Thread, which safely prints it to the console.

4. King Tower Destroyed! (Handling Signals):

- **The Game Rule:** Matches end abruptly when the main King Tower is destroyed, or if a player surrenders.
- **The OS Mapping:** We simulate this abrupt end via an OS Interrupt. If the user hits Ctrl+C on the keyboard, it sends a SIGINT signal to the process. Your server must intercept this signal using a custom handler, pause the game, print the total number of troops deployed during the match, and shut down gracefully.

Your server build is tied to your PES University developer ID (your SRN). Your unique SRN suffix dictates the mathematical rules of your specific server:

- **Arena Capacity (Buffer Size):** 5 + (SRN % 5) slots.
- **Max Elixir Capacity (Total Resources):** 10 + (SRN % 3) drops (A custom Over-Elixir game mode!).

Run the following boilerplate code clash_server.c, and fill in the logic for the aforementioned OS concepts in the To-Do sections mentioned. The appendix following this code, serves you with all the material to go ahead and code this yourself with ease. We recommend you to not rely on LLMs entirely, since just sitting with this material patiently can help enhance your skill and fundamentals in the given time-frame. We believe this will be a fruitful process that can help internalise these concepts in the best way possible :))

Appendix: The Developer's Spellbook (Reference Guide)

Welcome to the Supercell Backend Engineering docs. Use these conceptual hints and library references to implement the logic for your `TODO` sections.

TODO 1: Inter-Process Communication (IPC)

The Intuition: We need a way for multiple Card Spawner threads to send live text updates to a single Battle Logger thread. To do this, we use a unidirectional data channel. The Spawners will write to the "write end" of the channel, and the Logger will read from the "read end."

The Tool: `<unistd.h>`

- `int pipe(int pipefd[2]);`
  - This system call creates a pipe. It takes an array of two integers.
  - `pipefd[0]` becomes the file descriptor for the **read end**.
  - `pipefd[1]` becomes the file descriptor for the **write end**.
  - Returns `0` on success, or `-1` if it fails.

TODO 2: Deadlock Prevention (Elixir Management)

The Intuition: `available_elixir` is a globally shared variable. If multiple threads try to subtract from it simultaneously, your server will experience a race condition and the math will corrupt. Furthermore, to avoid deadlocks, a Spawner must lock the Elixir pool, check if it has enough Elixir to cover the *entire* cost of the card, deduct it if true, and then unlock the pool.

The Tool: `<pthread.h>`

- `pthread_mutex_lock(pthread_mutex_t *mutex);`
  - Acquires a lock. If another thread already owns it, the calling thread will pause and wait until it is unlocked.
- `pthread_mutex_unlock(pthread_mutex_t *mutex);`
  - Releases the lock so other threads can access the critical section.

TODO 3 & 4: Synchronization (The Bounded Buffer)

The Intuition: The Arena is an array with a strict limit (`arena_capacity`). You are implementing a classic **Producer-Consumer** pattern.

1. A Spawner (Producer) cannot deploy a troop if the Arena is full. It must wait for an "empty slot."
2. Deploying the troop into `arena[deploy_idx]` and updating `deploy_idx` is a *Critical Section*. Only one thread can do this at a time.
3. Once deployed, the Spawner must signal that a new "filled slot" exists in the Arena.

The Tools: `<semaphore.h>` **and** `<pthread.h>`

- `int sem_init(sem_t *sem, int pshared, unsigned int value);`
  - Initializes a semaphore. `pshared` should be `0` (shared between threads of the same process). `value` is the starting count. (Hint: Think about how many empty slots exist when the game starts vs. how many filled slots).
- `int sem_wait(sem_t *sem);`
  - Decrements the semaphore. If the value is `0` (e.g., no empty slots left), the thread goes to sleep until the value goes up.
- `int sem_post(sem_t *sem);`
  - Increments the semaphore, potentially waking up a sleeping thread.

TODO 5: Signals (Handling the Rage Quit)

The Intuition: When a player hits `Ctrl+C` in the terminal, the Operating System generates a specific software interrupt called `SIGINT`. By default, this immediately kills the process. We want to override this default behavior so that when `SIGINT` is fired, the server routes the execution to our custom `match_over_handler` function, allowing us to print the final game stats before shutting down.

The Tool: `<signal.h>`

- `void (*signal(int signum, void (*handler)(int)))(int);`
  - *Don't let the syntax scare you!* You just call `signal(SIGNAL_MACRO, function_name);`
  - `signum`: The macro for the signal you want to catch (in this case, `SIGINT`).
  - `handler`: The name of the function you want to execute when the signal is caught.